AI+X 중급 프로젝트
중간점검발표

202310843 송예린
202310847 심영주
202310856 유세리

목차

1. 스마트폰 각속도/가속도 데이터를 이용한 acitivity 분류 모델 구현
2. 생성형 AI 답변 비교
3. 추후 보완할 점

1. 스마트폰 각속도/가속도 데이터를 이용한 acitivity 분류 모델 구현

```python
# 각 행동별로 중요도 추출 함수 정의
def get_feature_importance_for_activity(activity):
    y_activity = (y == activity).astype(int)  # 해당 활동이면 1, 아니면 0

    # 훈련 데이터와 타겟 데이터 분할
    X_train, _, y_train, _ = train_test_split(x, y_activity, test_size=0.3, random_state=42)

    # 모델 학습
    rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
    rf_model.fit(X_train, y_train)

    # 중요도 추출
    feature_importances = rf_model.feature_importances_

    # 중요도를 데이터프레임으로 반환
    fi_df = pd.DataFrame({
        'feature_name': x.columns,
        'feature_importance': feature_importances
    })
    return fi_df
```

```python
# 10. 각 활동에 대해 feature importance 계산
fi_dynamic = get_feature_importance_for_activity('WALKING')  # 예시로 WALKING에 대해 계산
fi_standing = get_feature_importance_for_activity('STANDING')
fi_sitting = get_feature_importance_for_activity('SITTING')
fi_laying = get_feature_importance_for_activity('LAYING')
fi_walking_up = get_feature_importance_for_activity('WALKING_UPSTAIRS')
fi_walking_down = get_feature_importance_for_activity('WALKING_DOWNSTAIRS')

# 11. fi_analysis.csv 파일 불러오기 (없다면 새로 생성)
fi_analysis = pd.read_csv('fi_analysis.csv')

# 12. 각 행동별 중요도 컬럼을 fi_analysis에 추가
fi_analysis['fi_dynamic'] = fi_dynamic['feature_importance']
fi_analysis['fi_standing'] = fi_standing['feature_importance']
fi_analysis['fi_sitting'] = fi_sitting['feature_importance']
fi_analysis['fi_laying'] = fi_laying['feature_importance']
fi_analysis['fi_walking_up'] = fi_walking_up['feature_importance']
fi_analysis['fi_walking_down'] = fi_walking_down['feature_importance']

# 13. 전체 활동에 대한 중요도 계산 (모든 행동을 포함한 피처 중요도)
fi_all = get_feature_importance_for_activity('ALL')  # 'ALL'은 모든 활동을 포함하는 경우
fi_analysis['fi_all'] = fi_all['feature_importance']
```

```python
# 14. 최종 결과를 fi_analysis_with_all_importances.csv로 저장
fi_analysis.to_csv('fi_analysis_with_all_importances.csv', index=False)
print("Feature importance saved to fi_analysis_with_all_importances.csv")
```

“피처중요도”를 특징으로 잡고
모델을 구현

jupyter fi_analysis_with_all_importances.csv Last Checkpoint: 4 months ago

File Edit View Settings Help

Delimiter: ,

	feature_name	fi_all	fi_dynamic	fi_standing	fi_sitting	fi_laying	fi_walking_up
1	tBodyAcc-mean()-X	0.0	0.00032654207769201193	0.0004500950666752768	0.00035256842793624806	0.00039577218695763 34	0.00036762401404607684
2	tBodyAcc-mean()-Y	0.0	6.190315833662476e-05	0.0007195827669131822	0.0005450971964050651	0.0002941274951351439	0.0006945119329230284
3	tBodyAcc-mean()-Z	0.0	0.00013680374192683538	0.00013148349877505224	0.0003617269651750381	7.78743113156142e-06	0.00033446744333381754
4	tBodyAcc-std()-X	0.0	0.002283058296864874	0.005706381585558399	0.001893899975317829	0.0006540277596628455	0.006789877722658562
5	tBodyAcc-std()-Y	0.0	0.00021149272330156274	0.0021159882437827176	0.00037133147138850145	0.0	0.00016726456128555905
6	tBodyAcc-std()-Z	0.0	0.0002387125238171819	0.00043398437121943545	0.0005505250635798901	0.00038418653211692746	0.002104604544373318
7	tBodyAcc-mad()-X	0.0	0.003709157364494993	0.005057050295505872	0.00172355789656053	0.0012164546905795567	0.0037215818950614736
8	tBodyAcc-mad()-Y	0.0	0.00048130099744794123	0.0007325131850514935	0.00017640231834187902	1.4033301246609017e-05	0.0003743606120908286
9	tBodyAcc-mad()-Z	0.0	0.0003148823623928341	0.0005569131986021747	0.0009082456616684817	1.2755038798125783e-05	0.003031747645681354
10	tBodyAcc-max()-X	0.0	0.005762615656660213	0.00020790537652388258	0.0010636475723527228	0.0014359528218275024	0.0018459777802932562
11	tBodyAcc-max()-Y	0.0	0.0	0.00030160513367240375	0.00045703611362755666	7.801817575611429e-05	0.00035716546143617426
12	tBodyAcc-max()-Z	0.0	0.00015230378671979056	0.0007739818434160199	0.0003664629623958297	1.2991464645603827e-05	0.0002911232559910672
13	tBodyAcc-min()-X	0.0	0.0011648105671104607	0.0007113345627989025	0.0005339061709079609	0.000682294581100824	0.0006910682865981865
14	tBodyAcc-min()-Y	0.0	0.00020266632466112004	0.0003338707898541751	0.00037446783035553473	0.0	0.0001662429326615437
15	tBodyAcc-min()-Z	0.0	0.00023395914381911894	0.0004979198235719698	0.0006978241561092037	0.0	0.0013531391200490115
16	tBodyAcc-sma()	0.0	0.0031628984922947444	0.0022662073671406015	0.00031270506561098326	0.00020349177699068667	0.0030019906948253383
17	tBodyAcc-energy()-X	0.0	0.007920489564338114	0.007211559126536212	0.0015224935512752715	0.00017974935947209395	0.0037962602796285253
18	tBodyAcc-energy()-Y	0.0	0.00010931470214118777	0.0023017918442240835	0.00021383126509368858	1.7598617811267592e-05	0.00023080430116948294
19	tBodyAcc-energy()-Z	0.0	0.0003688133650764847	0.0004318015595019532	0.00030089849136725344	0.001055558271904826	0.0012087900032872491
20	tBodyAcc-iqr()-X	0.0	0.000773055775741897	0.007110793530309318	0.002122185223833599	0.001216513858217067	0.0001703907697136012

feature importance를 특징으로 잡고 모델 구현

1. N을 20개로 설정

3. 기본 모델링

- 세부 요구사항
  - 최소 4개 이상의 알고리즘을 적용하여 모델링을 수행한다.
  - 각 알고리즘별로 전체 변수로 모델링, 상위 N개 변수를 선택하여 모델링을 수행하고 성능 비교를 한다.
  - (옵션) 알고리즘 중 1~2개에 대해서, 변수 중요도 상위 N개를 선정하여 모델링하고 타 모델과 성능을 비교.
    - 상위 N개를 선택하는 방법은, 변수를 하나씩 늘려가며 모델링 및 성능 검증을 수행하여 적절한 지점을 찾는 것이다.

feature importance를 특징으로 잡고 모델 구현

1. N을 20개로 설정

```python
# 상위 N개 feature만 뽑기 위한 함수
def select_top_n_features(X, feature_importance_df, n):
    top_features = feature_importance_df['feature_name'].iloc[:n].values
    return X[top_features]
```

```python
# 상위 N개 선택
n = 20
X_train_top = select_top_n_features(X_train, feature_importance_df, n)
X_val_top = select_top_n_features(X_val, feature_importance_df, n)
```

(1) 알고리즘1 : 로지스틱 회귀

```python
# 전체 feature 사용
algo1 = LogisticRegression(max_iter=1000, random_state=42)
algo1.fit(X_train, y_train)
pred1 = algo1.predict(X_val)

acc1 = accuracy_score(y_val, pred1)
print("알고리즘1 (Logistic Regression) 전체 정확도:", acc1)
```

```
알고리즘1 (Logistic Regression) 전체 정확도: 0.983857264231096
```

```python
# 상위 N개 feature 사용
algo1_top = LogisticRegression(max_iter=1000, random_state=42)
algo1_top.fit(X_train_top, y_train)
pred1_top = algo1_top.predict(X_val_top)
acc1_top = accuracy_score(y_val, pred1_top)
print("알고리즘1 (Logistic Regression) 상위 N개 정확도:", acc1_top)
```

```
알고리즘1 (Logistic Regression) 상위 N개 정확도: 0.7068819031435853
```

(2) 알고리즘2 : 랜덤 포레스트

```python
from sklearn.ensemble import RandomForestClassifier

# 전체 feature 사용
algo2 = RandomForestClassifier(random_state=42)
algo2.fit(X_train, y_train)
pred2 = algo2.predict(X_val)
acc2 = accuracy_score(y_val, pred2)
print("알고리즘2 (Random Forest) 전체 정확도:", acc2)
```

```
알고리즘2 (Random Forest) 전체 정확도: 0.9813084112149533
```

```python
# 상위 N개 feature 사용
algo2_top = RandomForestClassifier(random_state=42)
algo2_top.fit(X_train_top, y_train)
pred2_top = algo2_top.predict(X_val_top)
acc2_top = accuracy_score(y_val, pred2_top)
print("알고리즘2 (Random Forest) 상위 N개 정확도:", acc2_top)
```

```
알고리즘2 (Random Forest) 상위 N개 정확도: 0.7952421410365336
```

(3) 알고리즘3 : KNN

```
]: from sklearn.neighbors import KNeighborsClassifier

# 전체 feature 사용
algo3 = KNeighborsClassifier()
algo3.fit(X_train, y_train)
pred3 = algo3.predict(X_val)
acc3 = accuracy_score(y_val, pred3)
print("알고리즘3 (KNN) 전체 정확도:", acc3)
```

```
알고리즘3 (KNN) 전체 정확도: 0.9481733220050977
```

```
]: # 상위 N개 feature 사용
algo3_top = KNeighborsClassifier()
algo3_top.fit(X_train_top, y_train)
pred3_top = algo3_top.predict(X_val_top)
acc3_top = accuracy_score(y_val, pred3_top)
print("알고리즘3 (KNN) 상위 N개 정확도:", acc3_top)
```

```
알고리즘3 (KNN) 상위 N개 정확도: 0.7434154630416313
```

(4) 알고리즘4 : XGboost

```
]: from xgboost import XGBClassifier
   from sklearn.metrics import accuracy_score
   from sklearn.preprocessing import LabelEncoder

   # 레이블 인코딩
   le = LabelEncoder()
   y_train_encoded = le.fit_transform(y_train)
   y_val_encoded = le.transform(y_val)

   # 전체 feature 사용
   algo4 = XGBClassifier(random_state=42, eval_metric='mlogloss')
   algo4.fit(X_train, y_train_encoded)
   pred4 = algo4.predict(X_val)
   acc4 = accuracy_score(y_val_encoded, pred4)
   print("알고리즘4 (XGBoost) 전체 정확도:", acc4)
```

```
알고리즘4 (XGBoost) 전체 정확도: 0.9923534409515717
```

```
]: # 상위 N개 feature 사용
   algo4_top = XGBClassifier(random_state=42, eval_metric='mlogloss')
   algo4_top.fit(X_train_top, y_train_encoded)
   pred4_top = algo4_top.predict(X_val_top)
   acc4_top = accuracy_score(y_val_encoded, pred4_top)
   print("알고리즘4 (XGBoost) 상위 N개 정확도:", acc4_top)
```

```
알고리즘4 (XGBoost) 상위 N개 정확도: 0.8181818181818182
```

feature importance를 특징으로 잡고 모델 구현

1. N을 50개로 설정

feature importance를 특징으로 잡고 모델 구현

1. N을 50개로 설정

```
# 상위 N개 feature만 뽑기 위한 함수
def select_top_n_features(X, feature_importance_df, n):
    top_features = feature_importance_df['feature_name'].iloc[:n].values
    return X[top_features]
```

```
# 상위 N개 선택
n = 50
X_train_top = select_top_n_features(X_train, feature_importance_df, n)
X_val_top = select_top_n_features(X_val, feature_importance_df, n)
```

(1) 알고리즘1 : 로지스틱 회귀

```python
# 전체 feature 사용
algo1 = LogisticRegression(max_iter=1000, random_state=42)
algo1.fit(X_train, y_train)
pred1 = algo1.predict(X_val)

acc1 = accuracy_score(y_val, pred1)
print("알고리즘1 (Logistic Regression) 전체 정확도:", acc1)
```

```
알고리즘1 (Logistic Regression) 전체 정확도: 0.983857264231096
```

```python
# 상위 N개 feature 사용
algo1_top = LogisticRegression(max_iter=1000, random_state=42)
algo1_top.fit(X_train_top, y_train)
pred1_top = algo1_top.predict(X_val_top)
acc1_top = accuracy_score(y_val, pred1_top)
print("알고리즘1 (Logistic Regression) 상위 N개 정확도:", acc1_top)
```

```
알고리즘1 (Logistic Regression) 상위 N개 정확도: 0.9422259983007647
```

정확도 변화

0. 706 → 0. 942

(2) 알고리즘2 : 랜덤 포레스트

```python
from sklearn.ensemble import RandomForestClassifier

# 전체 feature 사용
algo2 = RandomForestClassifier(random_state=42)
algo2.fit(X_train, y_train)
pred2 = algo2.predict(X_val)
acc2 = accuracy_score(y_val, pred2)
print("알고리즘2 (Random Forest) 전체 정확도:", acc2)
```

```
알고리즘2 (Random Forest) 전체 정확도: 0.9813084112149533
```

```python
# 상위 N개 feature 사용
algo2_top = RandomForestClassifier(random_state=42)
algo2_top.fit(X_train_top, y_train)
pred2_top = algo2_top.predict(X_val_top)
acc2_top = accuracy_score(y_val, pred2_top)
print("알고리즘2 (Random Forest) 상위 N개 정확도:", acc2_top)
```

```
알고리즘2 (Random Forest) 상위 N개 정확도: 0.9634664401019541
```

정확도 변화

0. 795 → 0. 963

(3) 알고리즘3 : KNN

```
0]: from sklearn.neighbors import KNeighborsClassifier

    # 전체 feature 사용
    algo3 = KNeighborsClassifier()
    algo3.fit(X_train, y_train)
    pred3 = algo3.predict(X_val)
    acc3 = accuracy_score(y_val, pred3)
    print("알고리즘3 (KNN) 전체 정확도:", acc3)
```

```
알고리즘3 (KNN) 전체 정확도: 0.9481733220050977
```

```
1]: # 상위 N개 feature 사용
    algo3_top = KNeighborsClassifier()
    algo3_top.fit(X_train_top, y_train)
    pred3_top = algo3_top.predict(X_val_top)
    acc3_top = accuracy_score(y_val, pred3_top)
    print("알고리즘3 (KNN) 상위 N개 정확도:", acc3_top)
```

```
알고리즘3 (KNN) 상위 N개 정확도: 0.8912489379779099
```

정확도 변화

0. 743 → 0. 891

(4) 알고리즘4 : XGboost

```
]: from xgboost import XGBClassifier
   from sklearn.metrics import accuracy_score
   from sklearn.preprocessing import LabelEncoder

   # 레이블 인코딩
   le = LabelEncoder()
   y_train_encoded = le.fit_transform(y_train)
   y_val_encoded = le.transform(y_val)

   # 전체 feature 사용
   algo4 = XGBClassifier(random_state=42, eval_metric='mlogloss')
   algo4.fit(X_train, y_train_encoded)
   pred4 = algo4.predict(X_val)
   acc4 = accuracy_score(y_val_encoded, pred4)
   print("알고리즘4 (XGBoost) 전체 정확도:", acc4)
```

```
알고리즘4 (XGBoost) 전체 정확도: 0.9923534409515717
```

```
]: # 상위 N개 feature 사용
   algo4_top = XGBClassifier(random_state=42, eval_metric='mlogloss')
   algo4_top.fit(X_train_top, y_train_encoded)
   pred4_top = algo4_top.predict(X_val_top)
   acc4_top = accuracy_score(y_val_encoded, pred4_top)
   print("알고리즘4 (XGBoost) 상위 N개 정확도:", acc4_top)
```

```
알고리즘4 (XGBoost) 상위 N개 정확도: 0.9821580288870009
```

정확도 변화

0. 818 → 0. 982

알고리즘별 정확도 비교 표

	전체 변수 정확도	상위 N개 변수 정확도
로지스틱회귀	0.9839	0.9422
랜덤포레스트	0.9813	0.9635
KNN	0.9482	0.8912
XGBoost	0.9924	0.9822

feature importance가 아닌 다른 특징을 가지는 모델 구현을 해 봐야 할 필요성을 느낌

1.처음에 N을 20으로 설정

2. 근데 전체 정확도보다 너무 낮게 나옴

3. N을 50으로 변경

4. 전체 정확도와 비슷해짐

- 전체 특성을 사용한 모델은 데이터의 모든 정보를 포함하지만,
과적합이나 불필요한 잡음이 포함될 수 있음
→ 정확도는 더 높게 나옴

- 상위 N개 특성만 사용한 모델은 필요한 정보만을 활용하여
더 간결한 모델 구현 가능
→ **<u>성능 향상이나 계산 효율성</u>**에 도움이 될 수 있음

- N=20에서 성능이 낮아지는 이유는 상위 20개 특성만을 사용하여 중요한 정보가 누락되거나 충분히 반영되지 않아서 모델이 과소적합(underfitting)될 가능성이 있기 때문

추후 보완할 점

- 더 정확한 평가 지표를 위해서 정밀도 / 재현율 / f1 스코어 등 추가 구현할 예정
- features importance가 아닌 다른 특징을 가지는 모델 구현 필요
- n개의 feature를 추출했을 때 성능이 얼마나 유지되는지 실험하고 그래프로 그려서 확인해보기
- 스마트폰 센서 기본 모델링 개선 및 성능 향상
- 스마트폰 센서 단계별 모델링 완성

“딥러닝 적용”

설명 가능한 딥러닝을 적용한 사용자 행동인식에 관한 연구 = A study of human activity recognition with explainable deep learning

https://www.riss.kr/link?id=T155517

저자	김지우
발행사항	서울: 광운대학교 대학원, 2020
학위논문사항	학위논문(석사) – 광운대학교 대학원 , 전자공학과 , 2020.2
발행연도	2020
작성언어	한국어
DDC	621.381 판사항(23)
발행국(도시)	서울
형태사항	vii, 32 p.: 삽도, 표; 27 cm.
일반주기명	지도교수 : 양성현 참고문헌 수록
UCI식별코드	I804:11012-200000299905

Process-aware Human Activity Recognition

Jiawei Zheng, Petros Papapanagiotou, Jacques D. Fleuriot, Jane Hillston

School of Informatics, University of Edinburgh
jw.zheng@ed.ac.uk, petrospapapan@gmail.com, jdf@ed.ac.uk, jane.hillston@ed.ac.uk

“사용자 행동 인식 관련 논문”

Abstract

ıs naturally follow distinct patterns when conducting
aily activities, which are driven by established prac-
nd processes, such as production workflows, social
and daily routines. Human activity recognition (HAR)
ıms usually use neural networks or machine learning
ques to analyse inherent relationships within the data.

To address this limitation, the idea of incorporati
main knowledge or contextual information into ML a
methods has been investigated to improve the accura
interpretability (Guo et al. 2023). Humans inherently
distinct patterns in their daily activities, influenced by
lished best practices and workflows. These include p
tion processes, social norms and habitual routines, s

성능향상에 목적

2. 생성형 AI 답변 비교

생성형 AI 답변 비교

문제1. 사회문제형

AI와 빅데이터를 활용해 지방소멸 문제를 해결할 수 있는 혁신 아이디어 3개를 제안해줘.
각 아이디어는 실행 방법과 예상 효과를 포함해야 해.

문제2. 기술형

10평 원룸 공간에 최적화된 가구 배치 알고리즘을 만들고 시각화해줘

문제3. 코딩형

초등학생도 사용할 수 있는 AI 일기장 웹앱을 만들어줘.
일기를 쓰면 감정을 분석해서 이모지를 보여주고, 한 줄 응원을 해줘.

Claude
BY ANTHROPIC

Grok 3

Gemini

*딥시크는 개인정보 문제로 제외

문제1. 사회문제형

AI와 빅데이터를 활용해 지방소멸 문제를 해결할 수 있는 혁신 아이디어 3개를 제안해줘.
각 아이디어는 실행 방법과 예상 효과를 포함해야 해.

🧠 AI & 빅데이터 기반 지방소멸 해결 아이디어 10선

1. 🎯 인구이동 예측 플랫폼

- **실행 방법**:
  AI 모델로 지역별 인구 변화 및 이주 패턴을 예측 (예: 기상, 교육, 일자리 데이터 통합).
  지역소멸 위험도 heatmap 시각화 제공.
- **예상 효과**:
  선제적 정책 대응 가능 → 예산 낭비 감소 + 위기 지역 조기 대응

2. 🏠 지역 맞춤형 청년 정착 AI 컨설팅

- **실행 방법**:
  개인의 관심사·직업·라이프스타일 데이터를 기반으로, AI가 적합한 지방 도시 추천.
  이사, 직장 연결, 주거 지원 정보까지 매칭 제공.
- **예상 효과**:
  청년층의 자발적 유입 유도, 지방 주거 만족도 상승

3. 📊 지방 일자리 빅데이터 매칭 서비스

- **실행 방법**:
  지역 중소기업의 숨겨진 채용 수요를 크롤링하고, AI로 유망 직군을 추천
  리모트/하이브리드 포함된 매칭 시스템 제공
- **예상 효과**:
  지방 일자리 미스매치 해소, 구직자 유입 촉진

무엇이든 물어보세요

+ 검색 심층 리서치 ···

ChatGPT는 실수를 할 수 있습니다. 중요한 정보는 재차 확인하세요.

ChatGPT

지방소멸 문제를 해결하기 위해 AI와 빅데이터를 활용한 혁신적인 아이디어를 아래와 같이 제안합니다. 각 아이디어는 실행 방법과 예상 효과를 포함합니다.

1. AI 기반 지역 맞춤형 일자리 매칭 플랫폼

- **실행 방법**: AI 알고리즘을 활용해 지역 주민의 스킬셋, 경력, 관심사를 분석하고, 빅데이터를 통해 지역 내 기업 및 원격 근무 가능한 일자리 정보를 수집해 최적의 매칭을 제공. 지방자치단체와 협력해 플랫폼을 구축하고, 사용자 데이터를 실시간으로 업데이트.
- **예상 효과**: 지역 인재 유출 방지 및 일자리 창출로 인구 감소를 완화. 예를 들어, 1년 내 5,000명 이상의 지역 주민이 새로운 일자리를 찾을 가능성.

2. 빅데이터 기반 지역 수요 예측 및 창업 지원 시스템

- **실행 방법**: 지역 소비 패턴, 인구 통계, 트렌드 데이터를 분석해 유망 사업 아이템을 도출. AI가 초기 창업자들에게 사업 계획 수립, 자금 조달 가이드를 제공하며, 지방정부와 연계해 세제 혜택 및 공간 지원.
- **예상 효과**: 소규모 창업 증가로 지역 경제 활성화. 연간 100개 이상의 신규 사업체 설립 가능.

3. AI 드론을 활용한 농업 자동화 및 유통 최적화

- **실행 방법**: AI 드론으로 농지 상태를 모니터링하고, 빅데이터로 날씨, 수확량 예측을 분석해 농업 생산성을 높임. 수확물의 유통 경로를 최적화하는 AI 시스템 도입.
- **예상 효과**: 농업 생산량 20% 증가 및 유통 비용 15% 절감으로 농촌 지역의 소득 증대와 인구 유지.

Grok이 어떻게 도와드릴까요?

DeepSearch Think Grok 3

Grok3

문제1. 사회문제형

AI와 빅데이터를 활용해 지방소멸 문제를 해결할 수 있는 혁신 아이디어 3개를 제안해줘.
각 아이디어는 실행 방법과 예상 효과를 포함해야 해.

AI와 빅데이터를 활용한 지방소멸 문제 해결 혁신 아이디어 3가지

다음은 AI와 빅데이터 기술을 활용하여 지방소멸 문제 해결에 기여할 수 있는 혁신적인 아이디어 3가지와 각 아이디어의 실행 방법 및 예상 효과입니다.

1. 맞춤형 생활 환경 추천 및 정착 지원 플랫폼 구축

- **아이디어 개요:** AI 기반으로 개인의 라이프스타일, 선호도, 필요 사항 등을 분석하고, 빅데이터를 활용하여 지역별 일자리, 주거 환경, 교육 시설, 문화 시설, 의료 서비스 등 생활 인프라 정보를 매칭하여 최적의 정착 지역을 추천하는 플랫폼을 구축합니다. 또한, 이주 결정 시 필요한 행정 절차, 이사, 지역 사회 연결 등을 지원하는 통합 서비스 기능을 제공합니다.
- **실행 방법:**
  - **데이터 수집 및 분석:** 공공 데이터 (인구 통계, 지역 정보, 생활 시설 정보 등) 및 민간 데이터 (부동산 매물 정보, 구직 정보, 소셜 미디어 활동 데이터 등)를 수집하고 AI 알고리즘을 통해 분석합니다.
  - **개인 맞춤형 추천 시스템 개발:** 사용자의 관심사, 직업, 가족 구성원, 선호하는 생활 방식 등을 입력받아 AI가 분석하고, 빅데이터와 매칭하여 사용자에게 적합한 지역과 맞춤형 정보를 추천합니다.
  - **정착 지원 서비스 연계:** 이사, 주거 계약, 행정 절차, 지역 커뮤니티 연결 등을 위한 정보를 제공하고 관련 기관 또는 서비스와 연계합니다. 온라인 상담, 챗봇 지원 등 비대면 서비스도 제공합니다.
  - **지속적인 피드백 및 개선:** 플랫폼 이용자의 만족도 및 실제 이주 결과를 분석하여 추천 알고리즘 및 서비스 품질을 지속적으로 개선합니다.
- **예상 효과:**
  - **인구 유입 증대:** 잠재 이주자에게 맞춤형 정보를 제공하고 정착 과정을 효율적으로 지원하여 지방으로의 인구 유입을 촉진합니다.
  - **성공적인 정착률 향상:** 개인의 필요와 지역 환경의 적합성을 높여 이주자의 만족도를 높이고 장기적인 정착을 유도합니다.
  - **지역 경제 활성화:** 새로운 인구 유입은 지역 소비 증대, 일자리 창출 등 지역 경제 활성화에 기여합니다.
  - **데이터 기반 정책 수립 지원:** 플랫폼 운영을 통해 얻어지는 데이터를 활용하여 지역 특성에 맞는 인구 유입 정책 및 정착 지원 정책 수립에 활용할 수 있습니다.

2. 지역 특화 산업 육성 및 일자리 매칭 AI 시스템

- **아이디어 개요:** 각 지역의 특성과 잠재력을 분석하여 경쟁력 있는 특화 산업을 발굴하고 육성하는 데 AI와 빅데이터를 활용합니다. 또한, 지역 기업의 채용 정보와 구직자의 역량, 경험, 선호도를 분석하여 최적의 일자리를 매칭하는 AI 기반 시스템을 구축하여 지역 일자리 문제를 해결하고 경제 활성화를 도모합니다.
- **실행 방법:**
  - **지역 특성 분석:** 지역별 산업 구조, 자원, 문화, 관광 요소 등 다양한 데이터를 분석하여 성장 가능성이 높은 특화 산업 후보군을 발굴합니다.
  - **특화 산업 육성 전략 수립:** 발굴된 특화 산업을 중심으로 기술 개발, 인력 양성, 마케팅 전략 등을 포함하는 맞춤형 육성 전략을 수립합니다. AI는 시장 동향 분석, 성공 사례 분석 등을 통해 전략 수립을 지원합니다.
  - **일자리 매칭 시스템 개발:** 지역 기업의 채용 공고, 요구 조건과 구직자의 이력, 기술, 희망 직무 등을 AI가 분석하여 적합한 일자리를 추천하고 매칭합니다.
  - **직업 훈련 및 연계 지원:** 필요한 경우, 구직자에게 특화 산업 관련 직업 훈련 프로그램을 제공하고 취업과 연계될 수 있도록 지원합니다.
  - **성과 측정 및 개선:** 시스템 운영 결과를 분석하여 매칭 성공률, 취업 유지율 등을 평가하고 시스템을 지속적으로 개선합니다.
- **예상 효과:**
  - **지역 경제 자립 기반 강화:** 지역 특성에 맞는 고부가가치 산업 육성을 통해 외부 의존도를 줄이고 지역 경제의 자립 기반을 강화합니다.
  - **양질의 일자리 창출:** 특화 산업 육성은 지역 주민들에게 안정적인 소득을 제공하는 양질의 일자리를 창출합니다.
  - **지역 인재 유출 방지 및 유입 촉진:** 지역 기반 산업 발달은 청년층의 수도권 유출을 막고, 관련 분야 전문 인력의 지역 유입을 유도합니다.
  - **지역 브랜드 가치 향상:** 성공적인 특화 산업 육성은 지역 이미지를 제고하고 관광 산업 등 다른 분야에도 긍정적인 영향을 미칩니다.

Gemini

AI와 빅데이터 기반 지방소멸 해결 혁신 아이디어

1. 맞춤형 지역 일자리 매칭 플랫폼

- **실행 방법:** AI 알고리즘을 활용하여 개인의 skills, 경력, 선호도와 지역 기업의 인력 수요를 정확히 매칭
- **예상 효과:** 청년 및 중장년층의 지역 정착률 25% 향상, 일자리 미스매치 해소

2. 스마트 귀농·귀촌 지원 시스템

- **실행 방법:** 빅데이터 기반 개인별 농업 적성 분석, AI 맞춤형 농업 교육 및 컨설팅 제공
- **예상 효과:** 성공적인 귀농·귀촌 비율 40% 증가, 지역 인구 유입 촉진

3. 초개인화 지역 창업 지원 플랫폼

- **실행 방법:** 지역 경제 데이터, 소비 트렌드 분석을 통한 개인 맞춤형 창업 아이디어 및 컨설팅 제공
- **예상 효과:** 지역 스타트업 성공률 35% 향상, 지역 경제 활성화

Claude

생성형 AI 비교

	ChatGPT	Grok3	Gemini	Claude
형식	번호, 이모지, 실행 방법 & 효과	키워드 중심 짧은 문장, 아주 간결	문단형, 자세한 설명 포함	번호 중심 요약 문단, 실행 방법, 효과 명시
장점	• 시각적으로 보기 쉬움 • 예상 효과에 수치/정책 관점이 들어감	• 효과에 정량적 표현이 있음 (예: 생산성 20% 증가) • 실용성 강조	• 분석 기반, 인프라 중심의 깊이 있는 설명 • 다양한 정책·산업 요인 고려	• 구조가 깔끔하고 명확 • 수치적 효과 제공 (성공률 %, 유입률 등)
단점	기술적 세부 내용은 간단히 서술됨	• 설명이 비교적 짧고 추상적 • 비즈니스/정책적 깊이는 부족	조금 길고 구조가 복잡해 덜 직관적	다소 무난하고 창의적 측면은 적음

*기술형과 코딩형은 더 좋은 질문 연구 예정

감사합니다.

AI+X 중급 프로젝트 중간점검발표